

Imperial College London
Department of Earth Science and Engineering
MSc in Applied Computational Science and Engineering

Independent Research Project
Final Report

Solving the Wave Equation by Optimising a Discrete Loss

by

Luke Dinsdale

repository: <https://github.com/ease-ada-lovelace-2025/irp-ld2022>

Supervisors:

Dr Lluís Guasch Batalla

Dr Ben Moseley

Dr Carlos Cueto Mondejar

August 2026

Abstract

The Optimising a Discrete Loss (ODIL) framework has been demonstrated on the 1D wave equation and shown to outperform Physics Informed Neural Networks, but its extension to higher dimensional problems and its comparison against conventional numerical solvers has not been examined. This work solves the forced acoustic wave equation in two dimensions using ODIL, and compares the results to finite-difference-based time stepping on accuracy, computational performance, and the treatment of known sources of error in wave modelling. The ODIL solution matches the accuracy and dispersive characteristics of time stepping, and destabilises beyond the Courant-Friedrichs-Lewy limit, though by in a different manner to time stepping that is postulated to result from propagation of a dominant unstable mode. ODIL's time and space complexities are less optimal than those of time stepping, but the preconditioner used is embarrassingly parallel, and exploiting this closes much of the performance gap. To support this work, a modular, extensible Python library was developed, which is benchmarked against analytical solutions and dispersion relations, and against Devito, a highly optimised finite differencing engine.

Acknowledgements

I would like to thank my supervisors, Dr. Lluís Gausch, Dr. Ben Moseley, and Dr. Carlos Cueto, for offering their time, support, and expertise to me during the IRP period. I feel very fortunate to have been able to contribute to the excellent work going on at Sonalis.

I would like to thank also the other students in the research group: Milica Pejic and Anton Belov, who contributed to the early development of my Python package, and provided many a conversation that helped me to navigate this challenging research space.

I acknowledge my use of AI tools, including Claude (<https://claude.ai/>) and Perplexity (<https://www.perplexity.ai/>), throughout the IRP period. Perplexity was used for exploring the literature around my topic of research, and Claude for programming guidance, including architecting the software package, debugging Python code, and prototyping plots for figures. I can declare however that the work submitted is my own, and reflects my own understanding of the topics addressed in this work.

1 Introduction

While an analytical solution to the wave equation exists under certain assumptions, practical problems require the flexibility to represent complex geometries, boundary conditions, and forcing terms, so the wave equation is instead solved numerically by approximating it with a discrete form. The finite difference [31, 9] and finite element [35] methods are the standard approaches, producing systems of equations that are solved either iteratively [23] or directly [19]. Both incur a computational cost that can be substantial, owing to the often ill-conditioned and sparse nature of the discretised systems [29, 26, 18].

Beyond cost, finite difference schemes for the wave equation carry their own characteristic errors. Discretisation introduces numerical dispersion and dissipation [3], causing waves to propagate at an incorrect speed, and solving the wave equation by time stepping can cause these errors accumulate both with propagation distance and time [2]. Suppressing them typically demands finer discretisation [19], which drives the computational cost up further. High order methods [8], dispersion reduction schemes [14, 5], and spectral methods [6] each address one side of this trade off, but the tension between accuracy and cost persists.

An alternative approach to finite difference based time stepping (FDTD) is to reformulate the problem as an optimisation problem. Optimising a Discrete Loss (ODIL) [25] can approximate the solution to a PDE by means of minimising a functional that encapsulates the discretised PDE and relevant constraints. In the functional, the PDE is expressed by a residual that is a function of the field of interest, which is optimised directly on the discretising grid. Thus, if a minimiser drives the loss to zero, it is the solution field which satisfies the discretised PDE and boundary conditions exactly. Unlike FDTD, ODIL need not enforce the PDE exactly at each timestep, and so its accuracy and stability properties may differ.

Karnakov et al. [25] demonstrated ODIL for PDE problems, including solving the homogeneous wave equation in one dimension, among others. The framework’s accuracy and computational cost were compared against those of Physics Informed Neural Networks (PINN), and ODIL was shown to outperform PINNs by several orders of magnitude. However, ODIL has not been demonstrated for two dimensional wave equation problems, nor has a comparison against FDTD been established. Furthermore, how ODIL treats known sources of errors, such as numerical dispersion, is yet to be studied.

This work aims to compare ODIL against FDTD for solving the two dimensional wave equation by addressing three defining Research Questions (RQ):

- RQ1: How does the accuracy of ODIL compare with that of FDTD?
- RQ2: How does the computational cost of ODIL compare to that of FDTD?
- RQ3: What are the dispersive and stability properties of ODIL?

In support of this work, a modular, extensible Python library has been developed. The library is benchmarked against Devito [28], a highly optimised library designed for the modelling of waves using FDTD.

2 Background

2.1 The Acoustic Wave Equation

Two dimensional wave propagation can be modelled by the linear acoustic wave equation. Assuming a lossless, isotropic medium, the pressure field $u = u(t, x, y)$ satisfies

$$\frac{\partial^2 u}{\partial t^2} - c^2 \nabla^2 u = s, \quad (1)$$

where $c = c(x, y)$ is the spatially varying wave speed, and ∇^2 denotes the two-dimensional Laplacian operator,

$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}, \quad (2)$$

for time t and the spatial coordinates (x, y) . $s = s(t)$ is some time dependent forcing term. For the purpose of this study, the source term was a time-shifted Ricker wavelet [30],

$$s(t, t_0) = (1 - 2\alpha) e^{-\alpha(t)}, \quad (3)$$

$$\alpha = \pi^2 f_0^2 (t - t_0)^2, \quad (4)$$

where f_0 is the peak frequency and t_0 is the time shift that centres the pulse. This choice of forcing wavelet is arbitrary and does not change the validity of the results.

2.1.1 Initial and Boundary Conditions

The pressure field is assumed to be initially at rest, corresponding to zero initial pressure and zero initial particle velocity. As such, the following initial conditions are imposed

$$u(0, x, y) = 0 \quad (5)$$

$$\frac{\partial}{\partial t} u(0, x, y) = 0 \quad (6)$$

At the boundaries, second order Higdon [22] absorbing boundary conditions are imposed. In two dimensions, they read

$$\left(\frac{\partial}{\partial t} + c_1 \frac{\partial}{\partial n} \right) \left(\frac{\partial}{\partial t} + c_2 \frac{\partial}{\partial n} \right) u = 0, \quad (7)$$

where the normal derivative

$$\frac{\partial u}{\partial n} = \nabla u \cdot \mathbf{n}, \quad (8)$$

is the spatial derivative of the pressure field in the direction perpendicular to the boundary with $\mathbf{n}$ as the outward unit normal. The parameters c_1 and c_2 represent the propagation speeds of outgoing waves, and are defined as

$$c_n = c \cos(\theta_n), \quad (9)$$

where θ_n is the angle of incidence relative to the boundary normal.

In Appendix A, it is shown that Equation (7) can be written as

$$\frac{\partial^2 u}{\partial t^2} + (c_1 + c_2) \frac{\partial^2 u}{\partial t \partial n} + c_1 c_2 \frac{\partial^2 u}{\partial n^2} = 0. \quad (10)$$

where mixed partial derivatives have been combined using the commutativity of partial differentiation. This form is used because it is more simple to discretise, since each derivative can be evaluated independently according to predefined finite difference operators.

3 Finite Difference Discretisation

The computational domain is discretised using a uniform Cartesian grid with N_t , N_x , N_y grid nodes in the temporal and spatial directions, respectively. The spatial coordinates are defined accordingly as

$$x_i := i\Delta x, \quad y_j := j\Delta y \quad (11)$$

and the temporal coordinates

$$t_n := n\Delta t, \quad (12)$$

with $n = 0, 1, \dots, N_t - 1$ (and analogously in space). Δx , Δy , and Δt denote the spatial and temporal grid spacings, respectively.

Under this discretisation, the discrete pressure field $u_{i,j}^n$ becomes an approximation of the continuous pressure field u :

$$u_{i,j}^n \approx u(t_n, x_i, y_j). \quad (13)$$

The spatial and temporal derivatives in Equation (1) are discretised with standard centred finite differences. This discussion will focus on second order stencils,

$$\frac{\partial^2 u}{\partial t^2} \approx \frac{u_{i,j}^{n+1} - 2u_{i,j}^n + u_{i,j}^{n-1}}{\Delta t^2} \quad (14)$$

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n}{\Delta x^2} \quad (15)$$

$$\frac{\partial^2 u}{\partial y^2} \approx \frac{u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n}{\Delta y^2} \quad (16)$$

however, higher order stencils have been implemented and are detailed in Appendix B.

3.1 Matrix Representation of the Discrete Operators

The pressure field is the optimisation variable in which all time levels are represented collectively in a vector,

$$\mathbf{u} := \left[(u^0)^T, (u^1)^T, \dots, (u^{N_t-1})^T \right]^T. \quad (17)$$

Here, the spatial indices are flattened into a single spatial degree of freedom index where $u^n \in \mathbb{R}^{N_x N_y}$ is the pressure field across space at the n -th time level. Correspondingly, the differential operators are naturally represented as matrices acting on the global vector.

The temporal second derivative operator is defined as $D_{tt} \in \mathbb{R}^{N_t \times N_t}$ and the second spatial derivative operators $D_{xx} \in \mathbb{R}^{N_x \times N_x}$ and $D_{yy} \in \mathbb{R}^{N_y \times N_y}$. In these matrices, rows corresponds to a point in space/time, whose nonzero entries identify the neighbouring points coupled into the stencil at that location. In the absence of boundary modifications, the finite difference derivative matrices have a near-Toeplitz structure, illustrated in Figure 1.

The discrete Laplacian operator $L_{xy} \in \mathbb{R}^{N_x N_y \times N_x N_y}$ is constructed using Kronecker products,

$$L_{xy} := D_{xx} \otimes I_y + I_x \otimes D_{yy}, \quad (18)$$

where $I_x \in \mathbb{R}^{N_x \times N_x}$ and $I_y \in \mathbb{R}^{N_y \times N_y}$ are identity matrices.

The global wave equation operator $A \in \mathbb{R}^{N_t N_x N_y \times N_t N_x N_y}$ is formed similarly by lifting the temporal operator to act across space, and the Laplacian operator to act across time, through Kronecker products:

$$A := D_{tt} \otimes I_{xy} - I_t \otimes (C^2 L_{xy}), \quad (19)$$

where $I_t \in \mathbb{R}^{N_t \times N_t}$ and $I_{xy} \in \mathbb{R}^{N_x \times N_y}$ are the identity matrices and $C \in \mathbb{R}^{N_x N_y \times N_x N_y}$ is a diagonal matrix containing the wave speed values on the computational grid. In the absence of boundary and initial conditions, A has a Block Toeplitz with Toeplitz Blocks (BTTB) structure.

Under this discretisation, the full system can be expressed as a linear system,

$$A\mathbf{u} = \mathbf{s}. \quad (20)$$

In ODIL, the optimisation variable comprises the entire discrete space-time pressure field, which motivates the described all-at-once formulation. Note, in practice the global operator is never explicitly formed. Instead, the action of A is applied through a series of matrix vector products with the constituent operator matrices, i.e. 'matrix free'.

3.2 Discretisation of the Initial and Boundary Conditions

Initial and boundary conditions are incorporated by modifying the corresponding rows of the discrete operator, such that the complete system represents both the PDE residual and constraints within a the same formulation. The initial pressure field is imposed directly by setting the solution at the initial time level to zero, and the zero initial velocity condition is enforced using a forward difference approximation of the first temporal derivative at $t = 0$. At the computational boundaries, a discrete approximation of the absorbing boundary condition is enforced by central difference approximations.

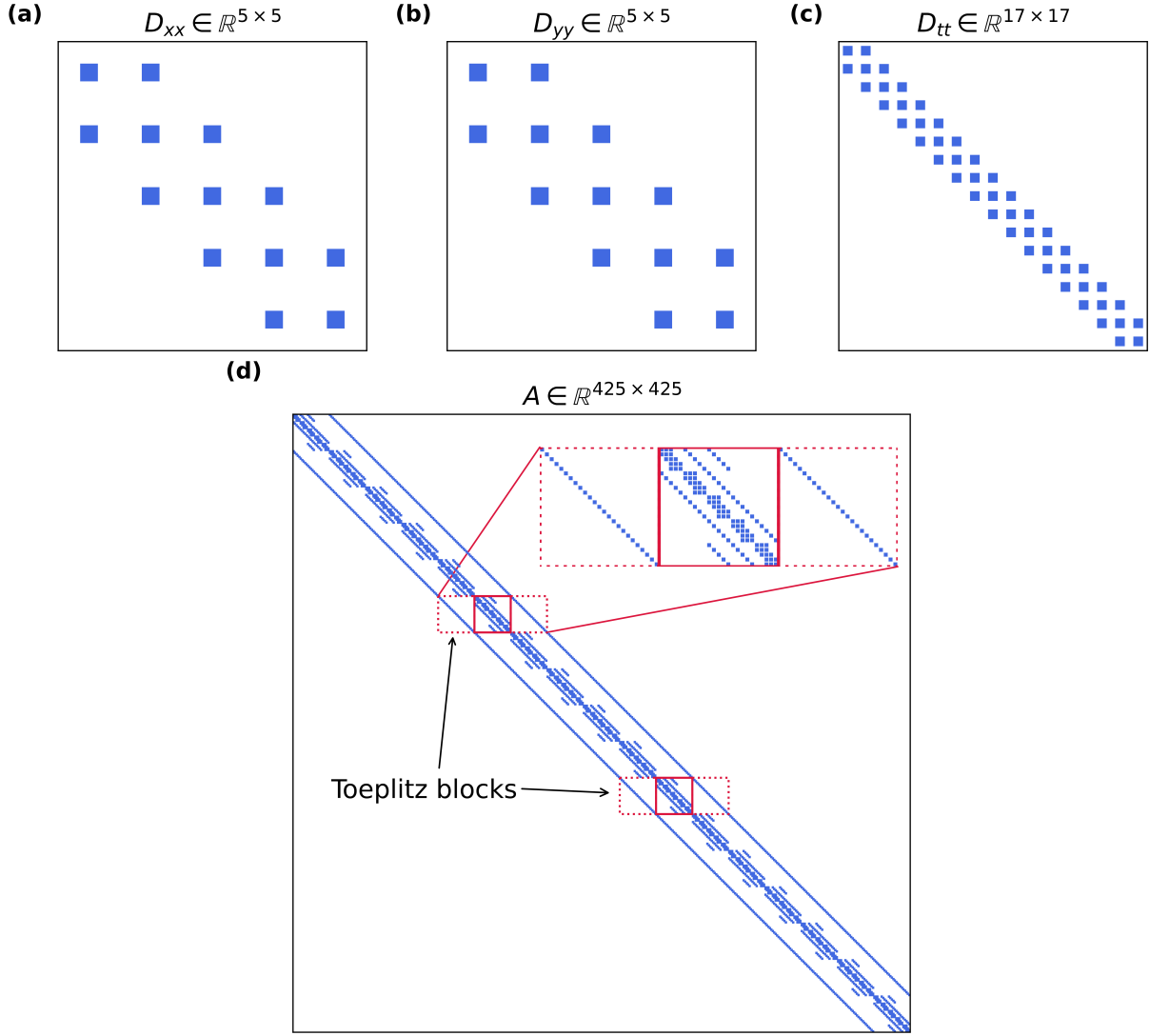


Figure 1: Sparsity plots of the operator matrices D_{xx} , D_{yy} , and D_{tt} (a - c) and the global operator matrix A (d) in the discretised wave equation for a second order centred difference (in the absence of boundary and initial conditions). Blue squares represent the non-zero entries in the matrices, and red boxes indicate the near-Block Toeplitz with Toeplitz Blocks structure of the global operator. The inset illustrates the block structure: off-diagonal blocks couple neighbouring time levels and the central block represents the current time with the spatial operator terms folded in.

Since a central difference stencil of order $2g$ requires g nodes either side of the evaluation point, the domain is extended by g ghost nodes on each side to avoid stencil truncation. Ghost values are populated via Lagrange extrapolation of values on the interior of the domain and are folded into a linear map G that is applied to the space-time vector. More detail on the construction of this linear map and incorporation of boundary conditions is provided in Appendix C.

3.3 Discrete Source Injection

Source injection in the discrete domain is handled using Hicks' method of Kaiser-windowed sinc interpolation [21]. This enables sources to be positioned at arbitrary physical coordinates rather than being restricted to the nearest grid node, thereby reducing grid positioning artefacts and improving the accuracy of source injection. Readers are encouraged to consult Hicks' work for

more information on the source injection methodology.

4 Optimising a Discrete Loss

ODIL solves an optimisation problem that seeks to minimise an objective function that enforces both the discretised PDE and its conditions. The discrete field is the optimisation variable, and the solution is obtained by minimising a loss function constructed from the residual of the discretised governing equations. For the two dimensional wave equation, the discrete residual can be written

$$r(\mathbf{u}) = A\mathbf{u} - \mathbf{s}. \quad (21)$$

The objective function $\mathcal{L}$ is the L2 norm of the discrete residual,

$$\mathcal{L}(\mathbf{u}) = \frac{1}{2} \|\mathbf{r}(\mathbf{u})\|^2. \quad (22)$$

It is written such that the residual tends to zero when the discrete wave equation is satisfied. The forward problem is therefore the unconstrained optimisation problem

$$\min_{\mathbf{u}} \mathcal{L}(\mathbf{u}). \quad (23)$$

Then it follows that the minimiser is a discrete pressure field $\mathbf{u}$ for which the residual is minimised, implying satisfaction of the discrete wave equation.

4.1 Solving the ODIL problem

As a baseline, this problem is solved using the L-BFGS-B method [38]. The remainder of this work discusses another method derived from the Gauss-Newton method, since plain gradient based optimisation was shown to lack practical performance characteristics (Section 6).

At the k -th iteration of the Gauss-Newton method, the solution field is updated according to

$$\mathbf{u}_{k+1} = \mathbf{u}_k + \delta u. \quad (24)$$

For nonlinear problems, the update variable δu is obtained by linearising the residual around the current iterate and finding the solution to the least squares problem that results through the normal equations. In Appendix D, it is shown that, since the residual is linear in $\mathbf{u}$, the normal equations reduce to $A\delta u = -r$.

The Gauss-Newton method converges in a single iteration for linear problems [4]. Substituting Equations 21 and 24 into the reduced normal equations yields

$$A(\mathbf{u}_{k+1} - \mathbf{u}_k) = -(A\mathbf{u}_k - \mathbf{s}) \quad (25)$$

$$A\mathbf{u}_{k+1} - A\mathbf{u}_k = -A\mathbf{u}_k + \mathbf{s} \quad (26)$$

$$\Rightarrow A\mathbf{u}_{k+1} = \mathbf{s} \quad (27)$$

Therefore, for the linear wave equation, applying the Gauss Newton update is equivalent to solving the discretised linear system directly. The distinction is primarily one of formulation:

instead of solving directly, the problem is posed as a least squares optimisation problem, with the Gauss-Newton method providing the linear solve.

GMRES is used to solve for $\delta \mathbf{u}$. GMRES [32] requires only matrix vector products, avoiding costly matrix factorisations and aligning with the aforementioned matrix free approach. In addition, GMRES solves $A\delta \mathbf{u} = -r$ directly, which avoids amplifying any ill-conditioning in the system, which would be the case for a least squares solve which introduces a transpose term that has the effect of squaring the conditioning number.

The GMRES algorithm displays poor convergence properties when the eigenvalues of the system are loosely clustered [34]. The all-at-once wave equation operator, prior to preconditioning, is strongly ill-conditioned, with poor clustering of its eigenvalues (Figure 2). Thus, there is motivation to precondition the linear system.

4.2 Preconditioning the Gauss-Newton Inner Solve

Karnakov et al. [25] suggest that ODIL can be accelerated using multigrid methods. However, multigrid methods can perform poorly for hyperbolic problems like the wave equation [10, 11]. Instead, a circulant preconditioner based on the ParaDiag-II methodology [16] is employed.

ParaDiag-II approximates the temporal Toeplitz operator by a circulant matrix, whose structure differs only in a wraparound block. Similarly, the resulting near-BTTB system is approximated by a block circulant matrix. Circulant matrices are diagonalisable by discrete Fourier transform (DFT) [20], so applying the preconditioner M^{-1} requires (i) an α -weighted Fast Fourier Transform (FFT) in time, (ii) N_t independent spatial solves in the Fourier domain, and (iii) an inverse FFT. Further details on the diagonalisation and α weighting are given in Appendix E.

Since the temporal operator is real valued, its DFT exhibits complex conjugate symmetry [24]. Consequently, for even N_t , only $(N_t/2) + 1$ distinct spatial factorisations are required. The remaining factorisations are obtained by complex conjugation. This has the effect of reducing the memory required to apply the preconditioner significantly.

The parameter $\alpha \in (0, 1)$ controls the wrap-around weighting and hence the conditioning of the preconditioned system [16]. A smaller α brings the circulant approximation closer to the original temporal operator, causing the eigenvalues of $M^{-1}A$ to cluster more closely around unity (Figure 2), drastically improving GMRES convergence. In all results presented here, $\alpha = 0.001$ is used.

The LU factorisation for each block corresponding to a frequency mode may be precomputed and cached in memory, but the caching strategy is not free. Since higher source frequencies require finer temporal grids, pre-caching creates a memory demand that grows sharply with frequency due to large factorisations that scale poorly in memory. To support higher frequency sources, a non-caching approach is implemented, where block factorisations are computed at each GMRES iteration and discarded. This costs runtime for repeated factorisations, but reduces memory constraints significantly.

The transformed circulant matrix decomposes into independent spatial systems, making the task of factorisation embarrassingly parallel. Hardware parallelism is exploited by distributing factorisations across processes, orchestrated by Python's `multiprocessing` module. This bypasses the Global Interpreter Lock and allows the $(N_t/2) + 1$ factorisations to run concurrently across CPU cores.

The temporal discretisation was fixed at second order for the following results. Extending to higher time order is a matter of generalising the factorisation and closing the additional initial conditions with more data, rather than a limitation of the preconditioning technique, and is left to future work. The spatial stencil order, unless otherwise stated, was sixth order.

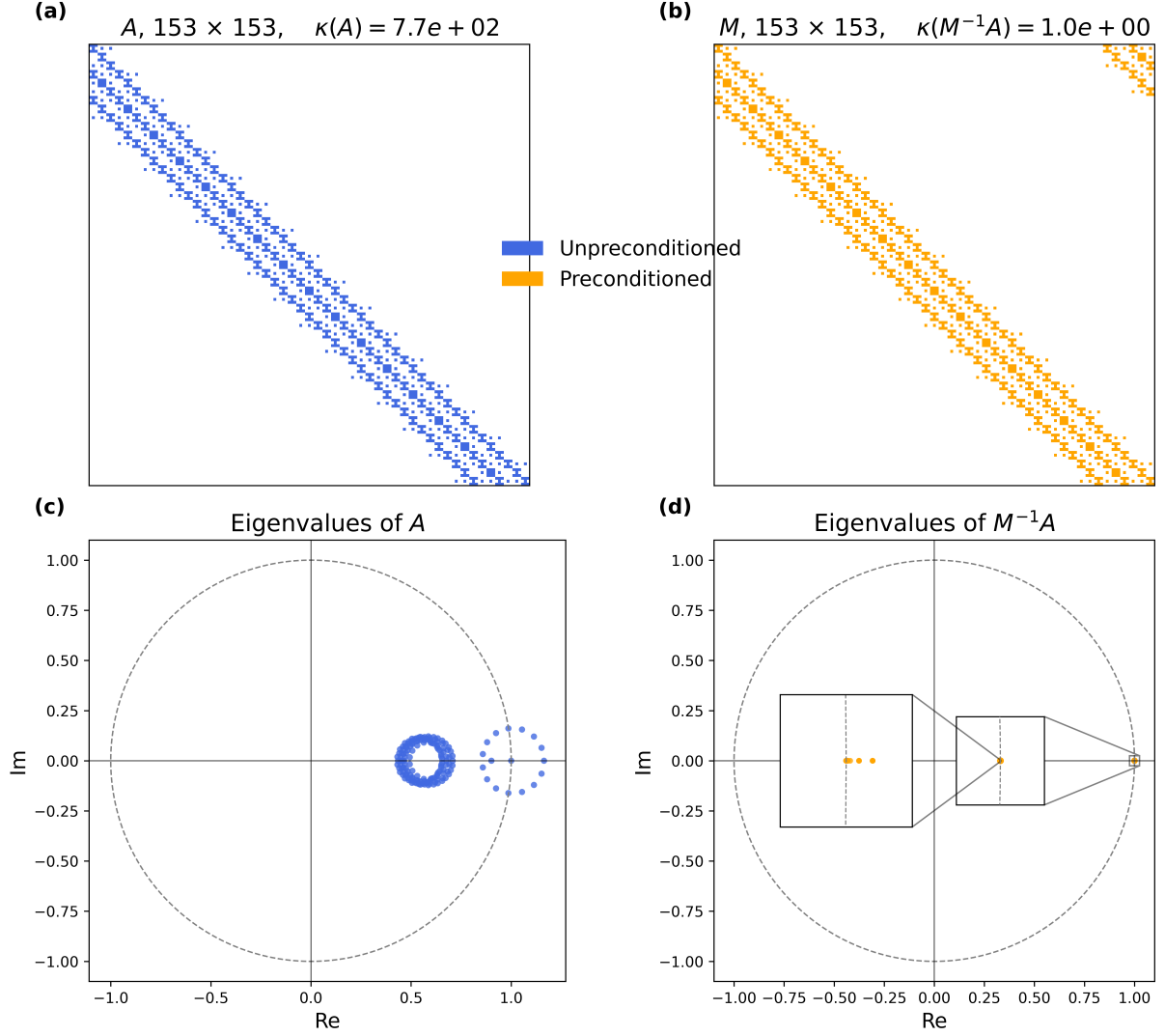


Figure 2: Panel (a): sparsity plot of the global operator matrix A (subject to the boundary and initial conditions described in Section 2.1.1). Panel (b): sparsity plot of the circulant approximation M , the preconditioner. Panels (c)/ (d): respective eigenvalue distributions. When the preconditioner is applied to the system, all eigenvalues have an absolute value within ± 0.0002 of unity.

5 Methods of Evaluation

The purpose of this study is to compare the characteristics of ODIL against FDTD solutions obtained using Devito [28] in three key areas: accuracy, performance, dispersion, and stability. This study focuses on the low-to-mid ultrasound band (50-200kHz), motivated by transcranial ultrasound applications.

ODIL and Devito solutions were obtained on the same grid and velocity model, with identical discretisation and physical parameters. ODIL components were built programmatically using the Python library developed for this research. FDTD components were built using elements of Devito’s `examples.seismic` submodule. Source injection and trace extraction for both sides of the problem was orchestrated using identical sinc interpolation methods (Section 3.3), verified against the Devito source code.

All relevant computational parameters are delineated on a per-experiment basis in Appendix F, and models and acquisition geometries are illustrated in Figure 3. The homogeneous model is used to compute analytical solutions of the wave equation, and the inclusion model isolates the effect of heterogeneity on solver scaling, while the Shepp-Logan phantom adds anatomical and geometric complexity.

All experiments were performed on an AMD Rome compute node from Imperial’s CX3 high performance computing facility. A node is comprised of two AMD EPYC 7742 processors, each with 64 physical cores and 128 threads, and 1Tb of RAM. Devito’s shared memory parallelism (orchestrated by OpenMP) is controlled through its Programmatic API for controlling environment variables [1]. All other variables, including compiler optimisations, are left to the default settings. For ODIL, the number of concurrent processes was set to be equal to the number of physical cores prescribed to particular experiments.

5.1 Accuracy Study

To address RQ1, the solutions achieved by ODIL and Devito are benchmarked against an analytical solution to the wave equation, for which the L2 norm of observed trace misfit is calculated at 24 receiver locations in the domain. Measuring at receivers represents realistic use cases, in which the full wave field is unknown and instead interpreted from discrete observations. It also avoids conflating numerical errors with boundary reflection artefacts.

Norms are calculated on traces normalised to unity to avoid being skewed by differences in wavelet scaling choices. Trace measurements are restricted to a window between the arrival times of the direct wave and any boundary reflections to prevent differing boundary reflection profiles from contaminating measurements. Measurements were made at a number of Degrees of Freedom (DOF) by varying the number of spatial grid nodes, and at each DOF measurements were repeated three times, with the mean value reported.

The analytical solution follows the definition of the Green’s function and convolution integral as in [37], and is calculated by convolving the Green’s function for the 2D wave equation with the Ricker wavelet. Appendix G demonstrates that the convolution integral can be written

$$u(r, t) = \frac{1}{2\pi} \int_0^{\cosh^{-1}(t/a)} s(t - a \cosh(\nu)) d\nu, \quad (28)$$

where $a = r/c$ is the time taken for a wave with speed c to move the source-receiver distance r , and ν is a dimensionless dummy integration variable. Rewriting in this way bypasses instability in the Green’s function (where the denominator can become singular), stabilising discrete integration. Gauss-Legendre quadrature is used to compute the discrete integral.

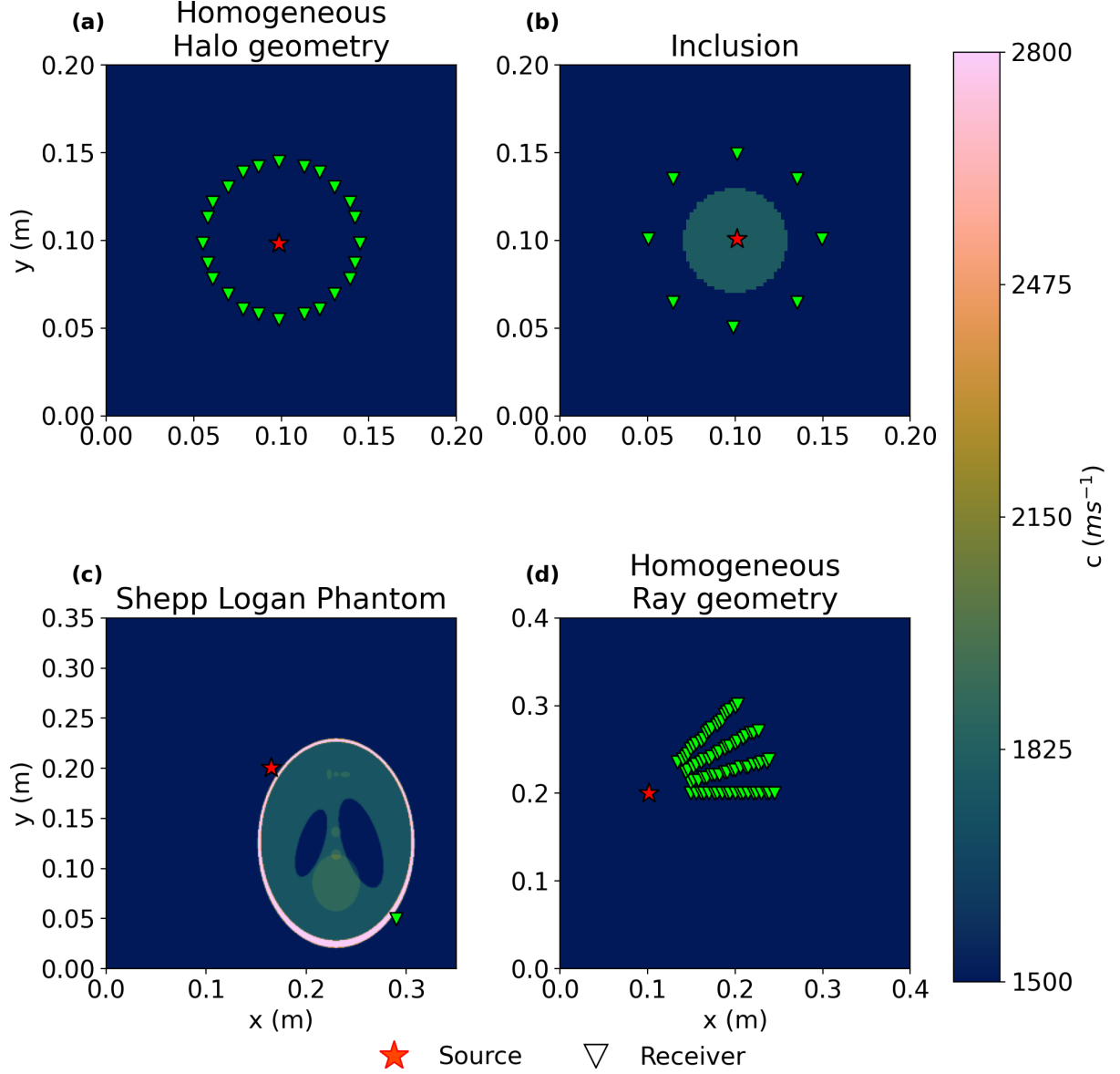


Figure 3: Illustrations of the wave speed models and acquisition geometries used across all experiments. The Shepp Logan model refers to the Shepp Logan Phantom [33]. The 'halo' geometry refers to the circular receiver layout, and similarly for the 'rays' geometry.

5.2 Performance Study

To address RQ2 concerning computational performance, the wall clock time to solution and memory consumption for both methods are measured on a small inclusion model (Figure 3b/ Table 2). Memory usage is reported as the Resident Set Size (RSS), computed using the standard Python library `resource`. The strong scaling behaviour of ODIL is determined for the caching and non-caching approaches, and parallel speedup is reported, defined as

$$S_n = \frac{T(n=1)}{T(n)}, \quad (29)$$

where T is the wall clock time, a function of n , the number of cores.

Reported timings represent the time required to obtain a converged solution, excluding one time initialisation and compilation overheads. For ODIL, this includes the time to compute the sparse factorisations of the preconditioner blocks described in Section 4.2. Devito is run once prior to each measurement to exclude JIT compilation overhead from the results. Each measurement was repeated three times, with the mean value reported.

Performance is first compared at the algorithmic level. This analysis is used to define bounds on the asymptotic complexity of each algorithm, which are used as reference points for understanding the runtime and memory performance of the ODIL and time stepping methodologies.

The operations that dominate the runtime and memory cost of the ODIL routine are the repeated sparse LU factorisations executed in the preconditioner, orchestrated by the SuperLU algorithm [12]. Exact space and time complexities for SuperLU are difficult to pin down, so measurements are compared to bounds from the literature. A conservative upper bound on the complexity is that of naive Gaussian elimination on an $n \times n$ dense matrix: $O(n^3)$ in time and $O(n^2)$ in space [27]. For SuperLU on a sparse matrix with partial pivoting and effective row ordering, a suitable lower bound was proposed by Alan G. and Esmond N. [17]: $O(n^{3/2})$ in time and $O(n \log n)$ in space.

For explicit time stepping, determining complexity is straightforward. In FDTD, for each grid point there is a fixed number of operations due to a fixed stencil width. For an $N_x \times N_y$ 2D spatial domain with $N_s = N_x \times N_y$ spatial nodes simulated over N_t time steps, the time complexity is therefore $O(N_s N_t)$. Time stepping approaches need not store the entire $N_s \times N_t$ wavefield, but can instead use rolling buffers (as Devito indeed does [28]), so the space complexity is merely $O(N_s)$.

5.3 Measuring Numerical Dispersion

The p - ω (slowness-frequency) transform method, described by Chekroun et al. [7], is used to measure phase velocity empirically. Receivers are positioned on rays relative to the source at a spacing much smaller than the minimum wavelength. Their locations are parametrised by a ray angle θ anticlockwise from the horizontal and a distance r along each ray (3d). At each receiver distance r_k along the propagation direction, the Fourier transform of the recorded signal is calculated, $s(r_k, \omega)$, where ω denotes angular frequency. Then, for a set of trial slowness values $p_i = 1/c_i$, $c_i \in [1300, 1600] \text{ ms}^{-1}$, each receiver's spectrum is phase shifted by $e^{i\omega p r_k}$, and spectra across receivers are summed to form a stack quantity $\hat{s}(p, \omega)$. When the trial slowness p matches the true slowness at that frequency, the phase corrected terms add constructively and $|\hat{s}(p, \omega)|$ is maximised, identifying the effective slowness. Sweeping over p for each ω and locating the maximum allows an empirical phase velocity to be calculated as

$$C_h(\omega) = \frac{1}{\hat{p}(\omega)}. \quad (30)$$

Measured phase velocities are compared to the analytical dispersion relation detailed by Alford, Kelly, and Boore [4]. For second order central differences, the result is obtained by substituting a plane wave solution for u into an explicit second order difference scheme, yielding

$$\sin^2 \frac{\omega \Delta t}{2} = \mathcal{C}^2 \left[\sin^2 \left(\frac{kh \cos \theta}{2} \right) + \sin^2 \left(\frac{kh \sin \theta}{2} \right) \right], \quad (31)$$

where $k = 2\pi/\lambda$ is the wavenumber corresponding to wavelength λ , and $h = \Delta x = \Delta y$ is the spatial grid spacing. $\mathcal{C}$ is the Courant number,

$$\mathcal{C} = \frac{c\Delta t}{h}. \quad (32)$$

This relation can be solved for ω , then the analytical phase velocity is given by $\tilde{C}_h = \omega/k$.

The measured and analytical phase velocities are determined for a number of grid resolutions. The resolution is parametrised by the number of Points Per Wavelength λ , defined as

$$\text{PPW} = \frac{\lambda}{\max(\Delta x, \Delta y)}. \quad (33)$$

5.4 Measuring Solution Stability

Solution stability is analysed at a range of CFL numbers. The CFL is given in 2D by

$$\text{CFL} = c\Delta t \sqrt{\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}}. \quad (34)$$

In general, for a second order explicit time stepping scheme to be stable the requirement is $C \leq 1$ [9], though the actual requirement is usually more stringent in practice.

Stability is assessed quantitatively and qualitatively by inspecting solutions on two models at a range of CFL numbers by varying temporal grid spacing. For the homogeneous model, $\text{CFL} \in [0.7, 9.2]$ was used, and for the Shepp Logan phantom model, $\text{CFL} \in [0.7, 17.2]$. At the upper limit of these bounds, the temporal sampling rate converges to the Nyquist limit.

On the homogeneous model, the analytical solution described in Section 5.1 is used to analyse error about the stability transition zone. Exponential decay is assumed, so the growth rate is reported as the slope of a straight line fitted to $\log(\max(|\mathbf{u}|))$ against t , with a positive value indicating growth. We also report the coefficient of determination R^2 for the fitted line. The Shepp Logan Phantom [33] is used to qualitatively infer how the solution degrades beyond the CFL limit.

6 Results and Discussion

As a baseline, L-BFGS-B, unpreconditioned GMRES, and preconditioned GMRES were run for a small homogeneous problem, measuring error norm, wall clock time, error norm, and reason for exit. The unpreconditioned GMRES method exited almost immediately on a relative tolerance criterion, indicating the solve is not viable without preconditioning, while L-BFGS-B achieved a significantly smaller error norm, but at an infeasible iteration cost. Accordingly, only the caching and non-caching ODIL methodologies are compared against FDTD going forward.

The relevant numerical results represented by figures in the discussion that follows are provided in Appendix H, Table 3-7.

6.1 Accuracy Properties

Error norm measurements for ODIL are within $\sim \pm 0.01$ of those of the FDTD solution (Figure 4). This suggests that ODIL converges to the same solution as time stepping, as expected given both share the same discretisation scheme. The accuracy of both methods refines with DOF due to an increasing number of points per wavelength (Figure 4a). The error that remains represents misfit resulting from the discretisation, and errors such as numerical diffusion. The results validate that the preconditioned ODIL methodology is capable of solving the forced acoustic wave equation in two dimensions to the same degree of accuracy as a time stepping methodology. This closes RQ1.

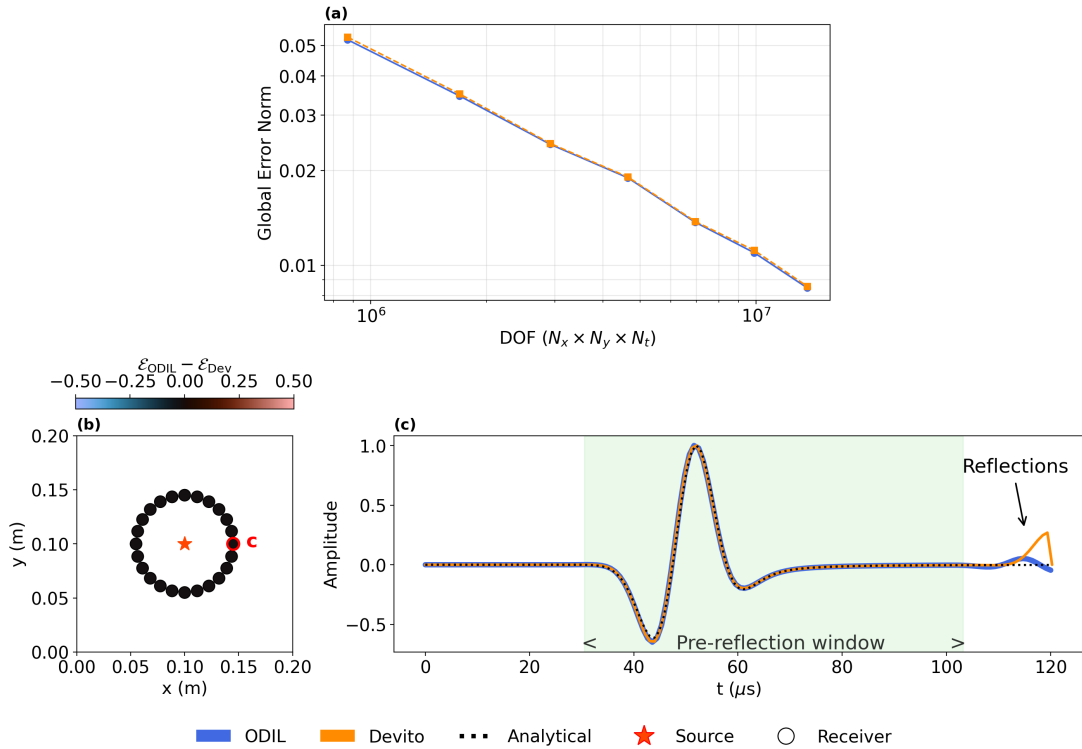


Figure 4: Panel (a) indicates global error norm as a function of Degrees of Freedom (DOF). Panel (b) illustrates the relative difference between the normalised trace residual norms of ODIL and Devito. Panel (c) shows waveforms observed at the receiver indicated by the red circle in Panel (b) against the analytical solution. The trace panel also demonstrates spurious boundary reflections and the pre-reflection window which omits them from metric calculations. Panels (b) and (c) visualise the results on the coarsest grid (lowest DOF) of $N_z = N_y = 80$, giving approximately 10 points per wavelength.

6.2 Computational Performance Properties

Both method’s run time and memory consumption scales within the theoretical time and space complexity bounds, as seen in Figure 5. Devito’s initial departure from the proposed complexities for time stepping is likely due to Python process overheads dominating measurements. Figure 5b demonstrates raw wall clock measurements and evidences that, when run in serial, ODIL underperforms FDTD, which is primarily attributable to inferior algorithmic scalability.

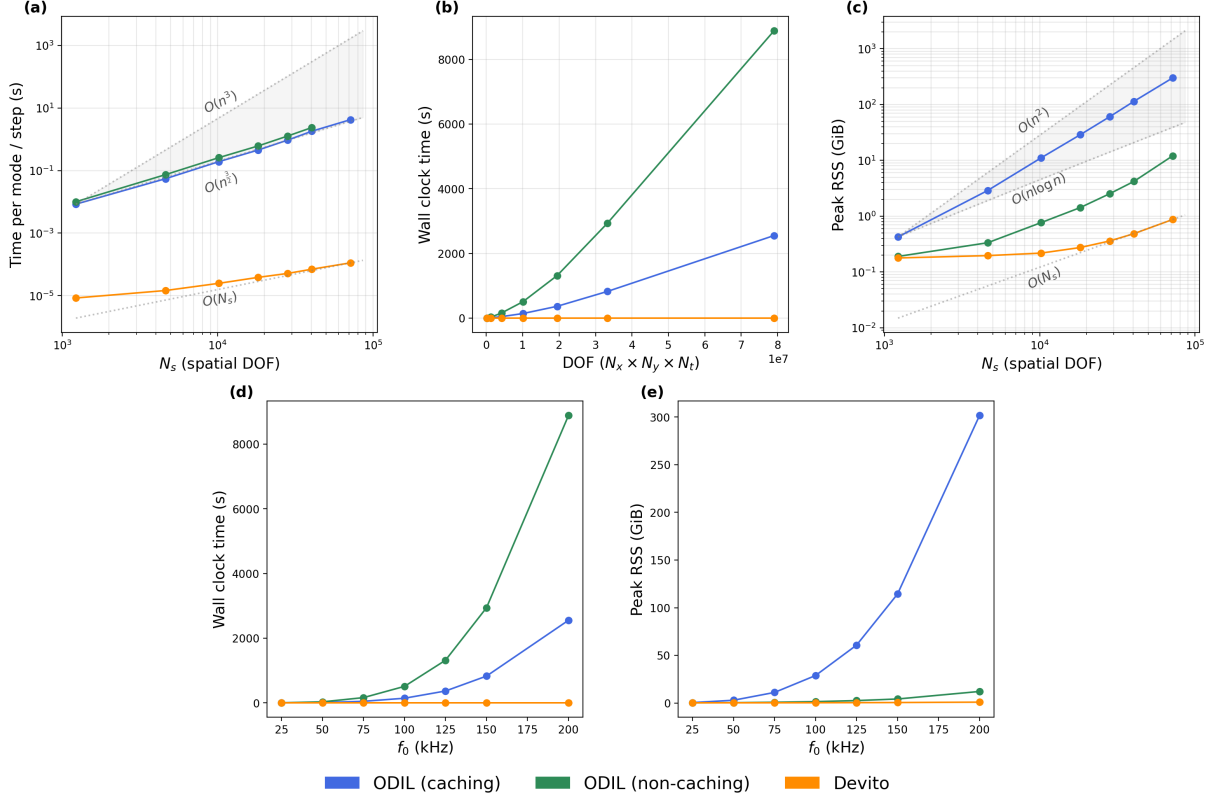


Figure 5: Serial (single core) runtime and memory performance behaviour, with comparisons to the discussed complexity bounds (signified by dashed lines/ grey regions).

In Figure 5b, the non-caching method sits below the lower bounds of memory complexity. Since factors are discarded after they are formed, the peak memory usage cannot exceed $M_{\max} = p \times M_k$, where p is the number of processes and M_k is the memory required to factorising a Toeplitz block corresponding to a single frequency mode. Figure 5c, isolates per mode factorisation wall clock cost and shows that run time scaling sits very close to the proposed lower bound on time complexity. This indicates near-optimal row ordering in the factorisations.

Figures 5c and d illustrate wall clock time and RSS as a function of peak source frequency f_0 , exposing a key limitation of the preconditioner. Temporal grid density depends on the peak frequency via the Nyquist criterion, and, since the ParaDiag II preconditioner scales poorly in memory when factorisations are cached, this limits the usable frequency range. Without sufficient RAM, at ultrasound frequencies the caching approach must be used instead, at the cost of an impractical factorisation time.

Figure 6a demonstrates near-linear speedup for ODIL up to 32 cores. Since the AMD Rome node used to execute the code has 4 NUMA domains, each formed of 32 cores, the departure from linear speedup is likely due to costly NUMA domain crosstalk. Devito shows negligible speedup, attributed to OpenMP overhead offsetting the modest gains available for a problem of this size. Figure 6c demonstrates how exploiting hardware parallelism allows ODIL to compete

more closely with time stepping, though the lowest solve duration is still ~ 10 times greater than that of Devito. Figure 6b demonstrates a less obvious idiosyncrasy of the non-caching methodology. Since $M_{\max}$ depends on the number of processes, when the number of processes is equal to the number of modes to be factorised, the memory consumption of the non-caching method approaches that of the caching method.

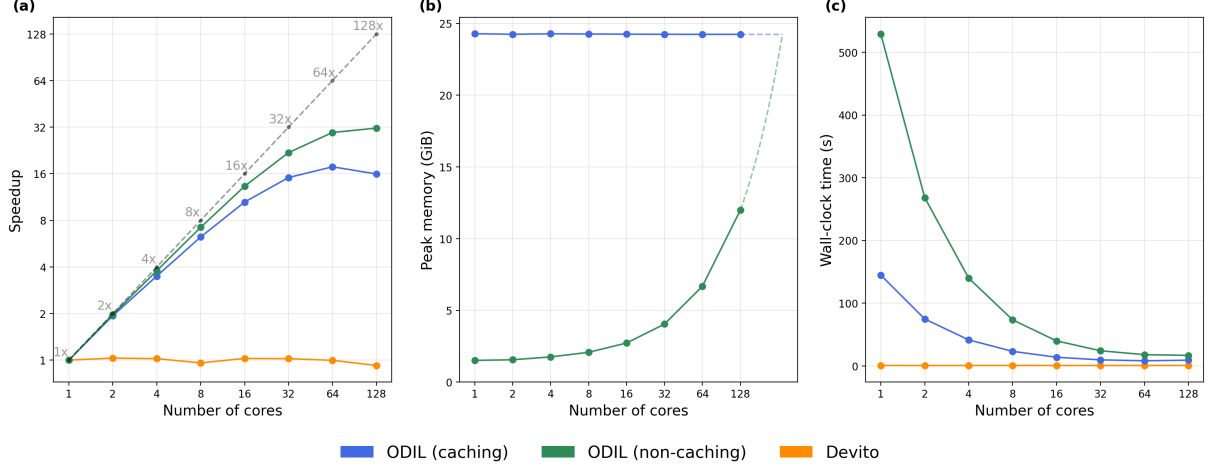


Figure 6: Strong scaling behaviour of (caching and non-caching) ODIL versus Devito: parallel speedup (a) and memory consumption as a function of the number of cores (b). Panel (c) illustrates raw wall clock times as a function of the number of processes. The dashed lines in panel (b) are extrapolated values demonstrating the convergence in memory consumption of the non-caching method to that of the caching method.

In summary, under the presented formulation, ODIL’s performance characteristics are less favourable than FDTD. This is primarily due to the usage of the ParaDiag II preconditioner, though its inherent parallelism closes the performance gap considerably, and alternative approaches remain unclear. It is important to note that this work was limited to shared memory, CPU bound computation. It is suggested that future work might concentrate on the following: other methods of preconditioning or alternative sparse factorisation algorithms, GPU acceleration, and distributed memory approaches. This closes RQ2.

6.3 Dispersion and CFL Effects

Figure 7 shows the percentage error in phase velocity versus as a function of Points Per Wavelength (PPW) versus the analytical dispersion relation. At all measured angles, ODIL and FDTD agree closely with the analytical relation across PPW levels, with the error converging toward zero as PPW increases. It is expected that ODIL and Devito display the same dispersive behaviour, since both discretise the wave equation with the same stencil. That ODIL's measured dispersion matches this analytical prediction provides further quantitative evidence that it converges to the same discrete solution as FDTD.

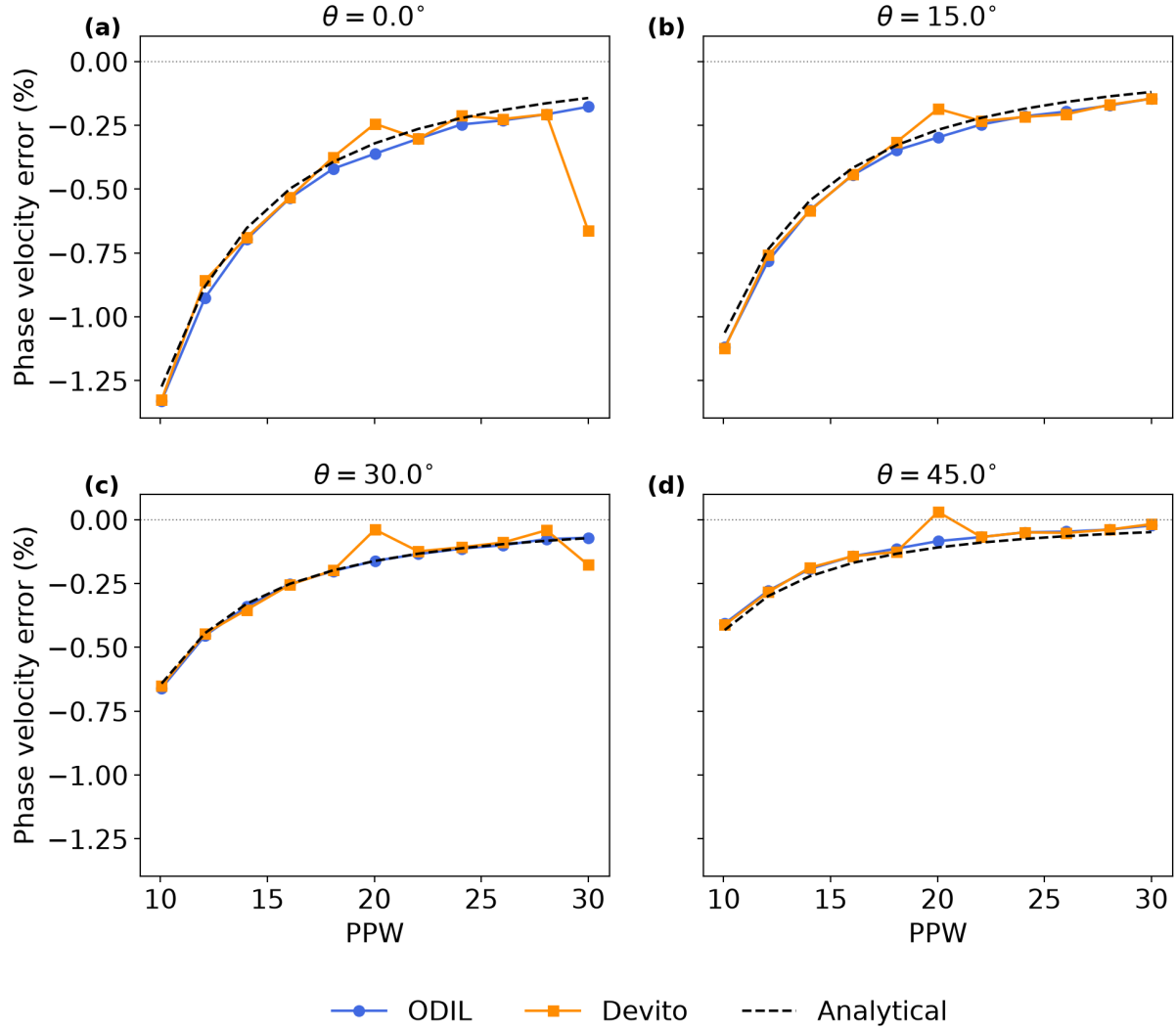


Figure 7: Percentage phase velocity error as a function of Points Per Wavelength (PPW) for the four ray angles relative to the horizontal.

Figure 8 establishes the behaviour of the ODIL and FDTD solutions in stable and unstable regimes. Below the classical CFL limit, ODIL's growth rate is approximately zero, indicating stability. Devito's growth rate is small and negative, indicating mild dissipation rather than instability, which is likely a boundary condition artefact, rather than a reflection of genuine differences in solution quality. Beyond the CFL limit, the FDTD solution rapidly destabilises, producing exponentially growing amplitudes that quickly become non finite. In contrast, ODIL remains finite, but is still subject to field growth over time, just in a different manner.

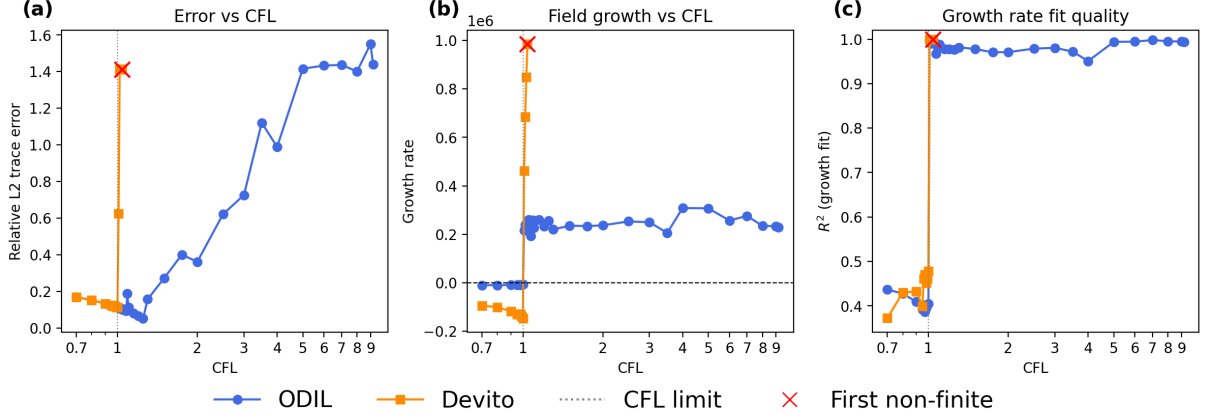


Figure 8: L2 trace norm error and growth rate of maximum field amplitude per timestep as a function of the CFL number. Red crosses indicate the CFL value at which the time stepping solution became non-finite. R^2 coefficients displayed in panel (c) show that the quality of fit is high (>0.9) for each point in the unstable regime, suggesting field growth is suitably captured by its metric.

Solutions on the Shepp Logan phantom model for both methodologies at stable and unstable CFL numbers are visualised in Figure 9. Within the stable regime, ODIL and time stepping solutions share similar peak amplitudes and wavelet shapes. Both solutions destabilise beyond the CFL limit, but in different ways. Devito solution fields grow exponentially, spilling into numerical overflow early in the simulation. On the contrary, ODIL initially propagate the source wavelet in a physically plausible manner, but at later times seemingly converges to a fixed, oscillatory pattern. That pattern resembles a two dimensional standing wave with a frequency that decreases with CFL number.

To further characterise this behaviour, the cosine similarity $s(t_i)$ and growth rate $r(t_i)$ between successive time levels is calculated for the ODIL solution at each CFL number according to

$$s(t_i) = \frac{\mathbf{u}_t \cdot \mathbf{u}_{t+1}}{\|\mathbf{u}_t\| \|\mathbf{u}_{t+1}\|}, \quad (35)$$

$$r(t_i) = \frac{\|\mathbf{u}_{i+1}\|}{\|\mathbf{u}_i\|}. \quad (36)$$

The cosine similarity approaches unity as time progresses for every CFL tested (Figure 10a), including those below the stability limit. This is a generic property of late time solutions rather than a signature of instability, and cannot solely describe the nature of the oscillatory pattern, but the per-step growth factor helps to separate the regimes (Figure 10b). Below the CFL limit, $r \approx 1.0$, indicating at most a slow decay consistent with energy redistribution through boundary absorption and geometric spreading. Above it, r settles to an approximately constant value that increases monotonically with CFL, from $r \approx 1.07$ at CFL = 1.3 to $r \approx 1.24$ at CFL = 17.2.

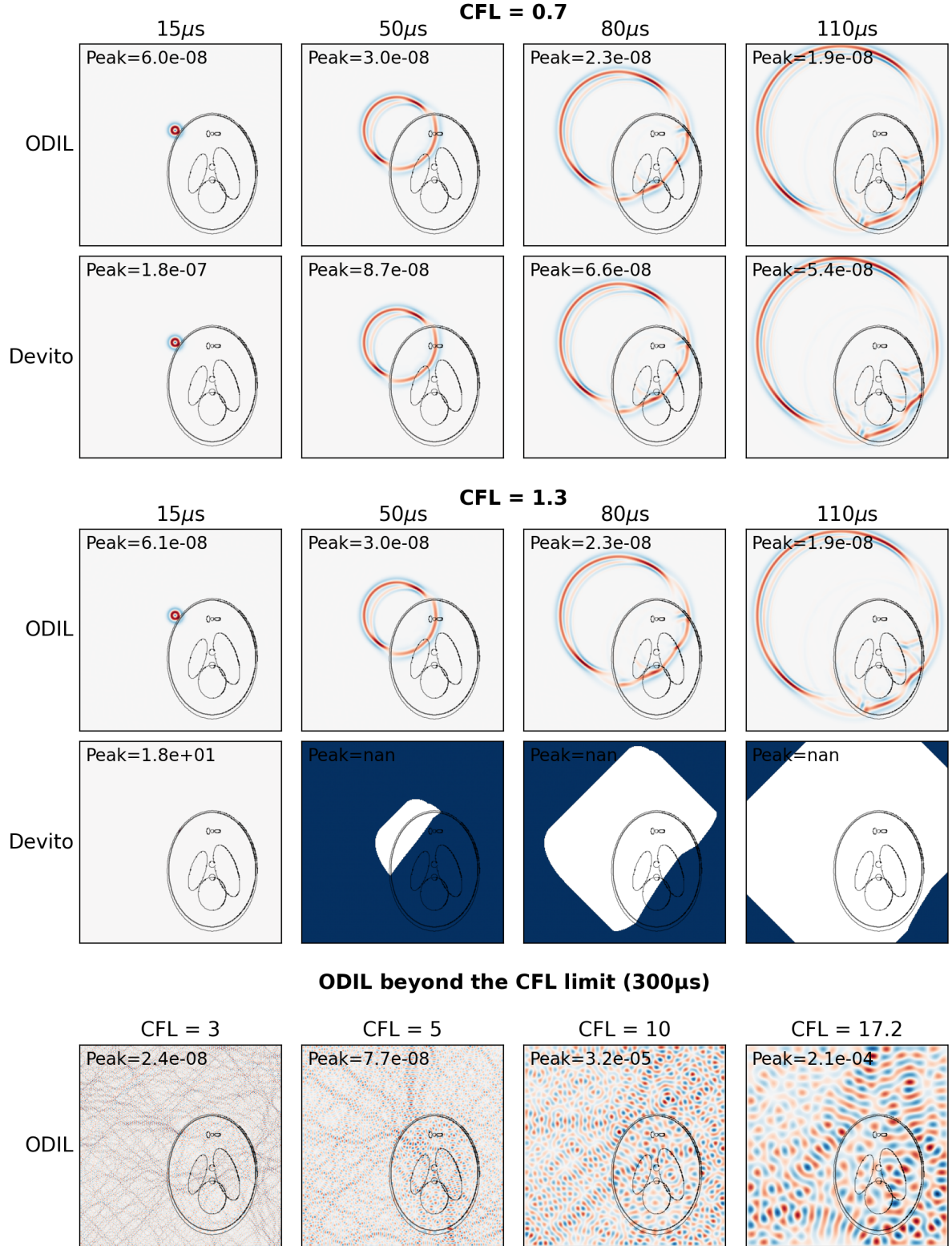


Figure 9: Wavefield snapshots through time on the Shepp-Logan phantom at different CFL numbers. Annotations indicate the peak amplitude per-frame. The final row indicates ODIL's late time convergence to a fixed, oscillatory mode resembling a two dimensional standing wave.

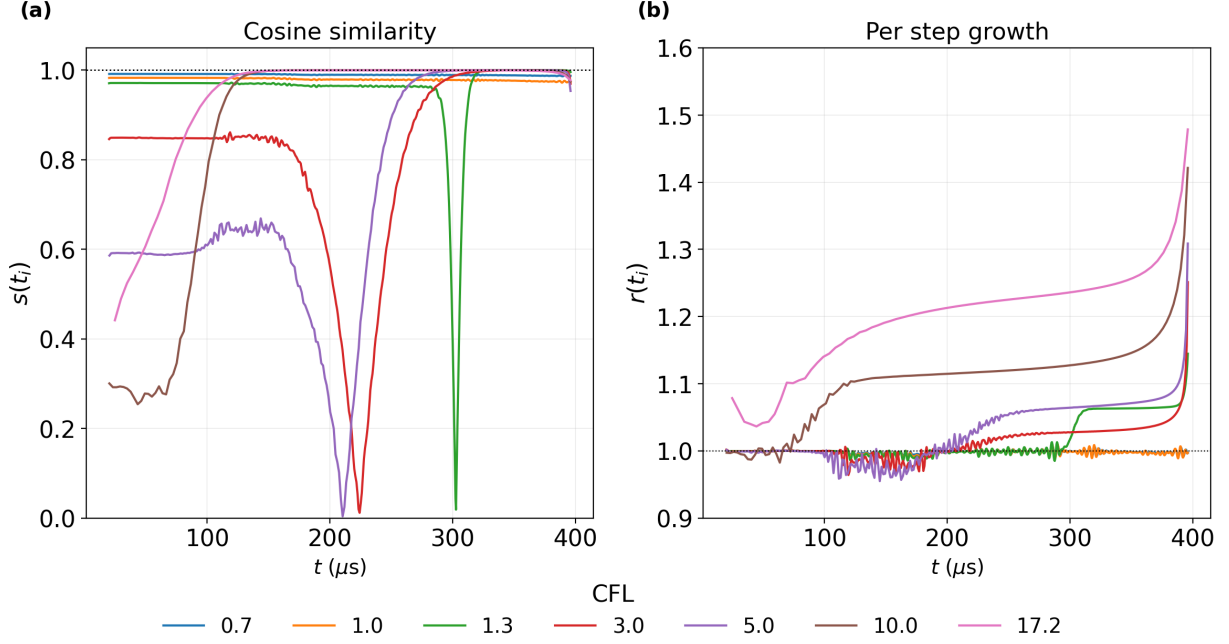


Figure 10: Cosine similarity $s(t_i)$ and per-step field growth $r(t_i)$ for the ODIL solutions on the Shepp Logan phantom model at different CFL numbers.

Jointly interpreting these metrics alongside the wavefield snapshots supports the idea of a single oscillatory mode. The amplitude remains non-zero, so there is a genuine field presence rather than simple numerical noise. The near constant late time cosine similarity suggests successive time levels share very similar spatial shapes, and the factor by which that shape is rescaled at each step, the growth rate, is approximately steady, bar some drift at the latest times. This is consistent with the notion of a single mode of oscillation that is repeatedly amplified through time.

In summary, beyond the CFL limit, the ODIL solution does not diverge in the manner of an explicit time stepper. Instead, it appears to converge onto a single spatial mode of oscillation at a rate set by the CFL number. It is postulated that this is because, beyond the CFL limit, ODIL propagates a dominant, unstable mode of oscillation, though there is insufficient evidence to fully verify this claim. Characterising the exact behaviour of the ODIL solution in the unstable regime and its mechanism is therefore left for future work. This closes [RQ3](#).

7 Conclusion

In this work, a methodology for solving the forced acoustic wave equation in two dimensions using the Optimising a Discrete Loss (ODIL) framework has been established and compared against traditional FDTD. ODIL was shown to inherit the accuracy properties of the underlying discretisation, converging to the same discrete solution as time stepping and matching it in dispersive character. The particular formulation presented, which uses an alpha circulant preconditioner, underperforms time stepping in memory usage and wall clock time when run in serial, owing to its higher computational complexity, though this gap narrows considerably when hardware parallelism is exploited. While many of the error characteristics are identical, the manner in which ODIL destabilises beyond the CFL limit was found to differ from FDTD. Evidence is presented that, in unstable regimes, ODIL settles on a single dominant mode of oscillation as time progresses, rather than diverging to complete numerical overflow. Identifying that mode, and the mechanism by which it is selected, is left to future work, and represents one of the most interesting avenue opened by this study. Under the formulation presented, ODIL's performance characteristics remain less favourable than those of FDTD. However, as hardware, software, and preconditioning methods continue to advance, the continued study of ODIL for such problems remains a valuable pursuit.

References

- [1] Configuration System | [xdslproject/devito](https://xdslproject.devito).
- [2] Well-posedness and Stability. In Bertil Gustafsson, editor, *High Order Difference Methods for Time Dependent PDE*, pages 13–68. Springer, Berlin, Heidelberg, 2008.
- [3] Numerical dispersion and dissipation. In Robert A. Marshall and Umran S. Inan, editors, *Numerical Electromagnetics: The FDTD Method*, pages 132–151. Cambridge University Press, Cambridge, 2011.
- [4] R. M. Alford, K. R. Kelly, and D. M. Boore. Accuracy of finite-difference modeling of the acoustic wave equation. *Geophysics*, 39(6):834–842, December 1974.
- [5] Yajun An. Uniform dispersion reduction schemes for the one dimensional wave equation in isotropic media. *Journal of Computational Physics*, 341:13–21, July 2017.
- [6] Christine Bernardi and Yvon Maday. Spectral methods. In *Techniques of Scientific Computing (Part 2)*, volume 5 of *Handbook of Numerical Analysis*, pages 209–485. Elsevier, January 1997.
- [7] Mathieu Chekroun, Loïc Le Marrec, Bruno Lombard, and Joël Piraux. Time-domain numerical simulations of multiple scattering to extract elastic effective wavenumbers. *Waves in Random and Complex Media*, 22(3):398–422, August 2012. arXiv:1202.3427 [physics.class-ph].
- [8] Gary C. Cohen. The Basic Equations. In Gary C. Cohen, editor, *Higher-Order Numerical Methods for Transient Wave Equations*, pages 3–13. Springer, Berlin, Heidelberg, 2002.
- [9] R. Courant, K. Friedrichs, and H. Lewy. On the Partial Difference Equations of Mathematical Physics. *IBM Journal of Research and Development*, 11(2):215–234, March 1967.
- [10] Hussam Al Daas, Niall Bootland, Tyrone Rees, and Sue Thorne. A Review of Time Stepping Techniques and Preconditioning for Hyperbolic and Anisotropic Elliptic Problems.
- [11] Federico Danieli and Andrew J. Wathen. All-at-once solution of linear wave equations.

- [12] James W. Demmel, John Gilbert, and Xiaoye S. Li. SuperLU Users' Guide. Technical Report UCB/CSD-97-944, May 1997.
- [13] Dean G. Duffy. *Green's Functions with Applications*. Chapman and Hall/CRC, New York, 2 edition, March 2015.
- [14] Bezalel Finkelstein and Raphael Kastner. Finite difference time domain dispersion reduction schemes. *Journal of Computational Physics*, 221(1):422–438, January 2007.
- [15] R. Fletcher. Newton-Like Methods. In *Practical Methods of Optimization*, pages 44–79. John Wiley & Sons, Ltd, 2000. Section: 3.
- [16] Martin J. Gander, Jun Liu, Shu-Lin Wu, Xiaoqiang Yue, and Tao Zhou. ParaDiag: parallel-in-time algorithms based on the diagonalization technique, May 2020.
- [17] Alan George and Esmond Ng. On the complexity of sparse \$QR\$ and \$LU\$ factorization of finite-element matrices. *SIAM Journal on Scientific and Statistical Computing*, 9(5):849–861, 1988. tex.eprint: <https://doi.org/10.1137/0909057>.
- [18] A. Gillman and P. G. Martinsson. A Direct Solver with \$O(N)\$ Complexity for Variable Coefficient Elliptic PDEs Discretized via a High-Order Composite Spectral Collocation Method. *SIAM Journal on Scientific Computing*, 36(4):A2023–A2046, January 2014.
- [19] S. Hao. Numerical Methods for Solving Linear Elliptic PDEs: Direct Solvers and High Order Accurate Discretizations. 2015.
- [20] M. Hariprasad and Murugesan Venkatapathi. Circulant decomposition of a matrix and the eigenvalues of Toeplitz type matrices. *Applied Mathematics and Computation*, 468:128473, May 2024.
- [21] Graham. J. Hicks. Arbitrary source and receiver positioning in finite-difference schemes using Kaiser windowed sinc functions. *Geophysics*, 67(1):156–166, January 2002.
- [22] Robert L. Higdon. Numerical absorbing boundary conditions for the wave equation. *Mathematics of Computation*, 49(179):65–90, 1987.
- [23] Ilse C. F. Ipsen and Carl D. Meyer. The Idea Behind Krylov Methods. *The American Mathematical Monthly*, 105(10):889–899, December 1998. _eprint: <https://doi.org/10.1080/00029890.1998.12004985>.
- [24] Steven G. Johnson and Matteo Frigo. A Modified Split-Radix FFT With Fewer Arithmetic Operations. *IEEE Transactions on Signal Processing*, 55(1):111–119, January 2007.
- [25] Petr Karnakov, Sergey Litvinov, and Petros Koumoutsakos. Solving inverse problems in physics by optimizing a discrete loss: Fast and accurate learning without neural networks. *PNAS Nexus*, 3(1):pgae005, January 2024.
- [26] Robert C. Kirby and Lawrence Mitchell. Solver Composition Across the PDE/Linear Algebra Barrier. *SIAM Journal on Scientific Computing*, 40(1):C76–C98, January 2018.
- [27] Xiaoye S. Li. An overview of SuperLU: Algorithms, implementation, and user interface. *ACM Transactions on Mathematical Software (TOMS)*, 31(3):302–325, September 2005.
- [28] Mathias Louboutin, Michael Lange, Fabio Luporini, Navjot Kukreja, Philipp A. Witte, Felix J. Herrmann, Paulius Velesko, and Gerard J. Gorman. Devito (v3.1.0): an embedded domain-specific language for finite differences and geophysical exploration. *Geoscientific Model Development*, 12(3):1165–1187, March 2019. arXiv:1808.01995 [cs.DM].

- [29] Prasad Pranjali and David J. Silvester. Optimal iterative solvers for linear nonsymmetric systems and nonlinear systems with PDE origins: Balanced black-box stopping tests, May 2018. Number: 2018.13.
- [30] Norman Ricker. Wavelet functions and their polynomials. *Geophysics*, 9(3):314–323, July 1944.
- [31] Milton E. Rose. Finite Difference Schemes for Differential Equations. *Mathematics of Computation*, 18(86):179–195, 1964.
- [32] Youcef Saad and Martin H. Schultz. GMRES: a generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 7(3):856–869, 1986. `tex.eprint`: <https://doi.org/10.1137/0907058>.
- [33] L. A. Shepp and B. F. Logan. The Fourier reconstruction of a head section. *IEEE Transactions on Nuclear Science*, 21(3):21–43, June 1974.
- [34] Lloyd N. Trefethen and David Bau, III. Chapter III: Conditioning and Stability. In *Numerical Linear Algebra*, Other Titles in Applied Mathematics, pages 89–144. Society for Industrial and Applied Mathematics, January 1997.
- [35] M. J. TURNER, R. W. CLOUGH, H. C. MARTIN, and L. J. TOPP. Stiffness and Deflection Analysis of Complex Structures. *Journal of the Aeronautical Sciences*, 23(9):805–823, 1956. `._eprint`: <https://doi.org/10.2514/8.3664>.
- [36] Stéfan van der Walt, Johannes L. Schönberger, Juan Nunez-Iglesias, François Boulogne, Joshua D. Warner, Neil Yager, Emmanuelle Goullart, and Tony Yu. `scikit-image`: image processing in Python. *PeerJ*, 2:e453, June 2014.
- [37] Kees Wapenaar. Multiple Reflections on Huygens’ Principle. *Geophysical Prospecting*, 73(6):e70038, 2025. `._eprint`: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/1365-2478.70038>.
- [38] Yunhai Xiao and Zengxin Wei. A new subspace limited memory BFGS algorithm for large-scale bound constrained optimization. *Applied Mathematics and Computation*, 185(1):350–359, February 2007.

Appendix

A Expansion of the Higdon Boundary Condition

Starting from the Higdon boundary condition (Equation (7)), the interior operator is defined as

$$v := \left(\frac{\partial}{\partial t} + c_2 \frac{\partial}{\partial n} \right) u, \quad (37)$$

using which the Higdon condition can be written

$$\left(\frac{\partial}{\partial t} + c_1 \frac{\partial}{\partial n} \right) v = 0. \quad (38)$$

Applying the outer operator,

$$\frac{\partial v}{\partial t} + c_1 \frac{\partial v}{\partial n} = 0, \quad (39)$$

and substituting v yields

$$\frac{\partial}{\partial t} \left(\frac{\partial u}{\partial t} + c_2 \frac{\partial u}{\partial n} \right) + c_1 \frac{\partial}{\partial n} \left(\frac{\partial u}{\partial t} + c_2 \frac{\partial u}{\partial n} \right) = 0. \quad (40)$$

Differentiating each term gives

$$\frac{\partial^2 u}{\partial t^2} + c_2 \frac{\partial^2 u}{\partial t \partial n} + c_1 \frac{\partial^2 u}{\partial n \partial t} + c_1 c_2 \frac{\partial^2 u}{\partial n^2} = 0 \quad (41)$$

and since mixed derivatives commute, this can be written

$$\frac{\partial^2 u}{\partial t \partial n} = \frac{\partial^2 u}{\partial n \partial t}, \quad (42)$$

and rearrange to find

$$\frac{\partial^2 u}{\partial t^2} + (c_1 + c_2) \frac{\partial^2 u}{\partial t \partial n} + c_1 c_2 \frac{\partial^2 u}{\partial n^2} = 0, \quad (43)$$

which is the result presented in Equation (10).

B Central Difference Schemes

On the discrete computational domain described in Section 3, the temporal and spatial derivatives in the wave equation (Equation 1) are approximated using standard central difference schemes of orders 2, 4, 6, and 8. For a $2P + 1$ point stencil in x , the general form of the centred difference is

$$\frac{\partial^2}{\partial x^2} u(t_n, x_i, y_j) \approx \frac{1}{\Delta x^2} \sum_{p=-P}^P c_p u_{i+m}, \quad (44)$$

which reads analogously in y and t . The coefficients c_p are obtained via the usual Taylor series construction and are chosen to achieve second, fourth, and eighth order accuracy. Those coefficients are tabulated below.

Order	Offsets	Coefficients
2nd	-1, 0, 1	1, -2, 1
4th	-2, -1, 0, 1, 2	$-\frac{1}{12}, \frac{4}{3}, -\frac{5}{2}, \frac{4}{3}, -\frac{1}{12}$
6th	-3, -2, -1, 0, 1, 2	$\frac{1}{90}, -\frac{3}{20}, \frac{3}{2}, -\frac{49}{18}, \frac{3}{2}, -\frac{3}{20}, \frac{1}{90}$
8th	-4, -3, -2, -1, 0, 1, 2, 3, 4	$-\frac{1}{560}, \frac{8}{315}, -\frac{1}{5}, \frac{8}{5}, -\frac{205}{72}, \frac{8}{5}, -\frac{1}{5}, \frac{8}{315}, -\frac{1}{560}$

Table 1: Coefficients for centred finite difference approximations of the second derivative.

C Matrix Form of the Boundary Operator

The described ghost node reconstruction is a linear operation. It can therefore be represented as a sparse matrix

$$\mathbf{u}_{\text{full}} = G\mathbf{u}, \quad (45)$$

where $G \in \mathbb{R}^{(n+2g) \times n}$ maps the n interior unknowns to the extended vector containing both interior and ghost nodes. The matrix G consists of an identity block corresponding to the interior nodes and additional rows containing the Lagrange extrapolation weights for the ghost nodes.

Accordingly, in the x direction,

$$D_x^{(g)} = D_x G, \quad D_{xx}^{(g)} = D_{xx} G, \quad (46)$$

which yields ordinary square difference operators, valid at every node including the boundary, with the extrapolated ghost contribution folded in. The discrete operators of the Higdon boundary condition are broadcasted to space-time by means of a Kronecker product.

Then, restricting to the n_b nodes on edge b with a selection matrix R_b , the discrete normal derivative operators can be written

$$D_n = R_b D_x^{(g)}, \quad D_{nn} = R_b D_{xx}^{(g)}, \quad (47)$$

with $D_x^{(g)}$ replaced by $D_y^{(g)}$ on the horizontal edges, and where $\cdot^{(g)}$ denotes that the operator acts on boundary nodes. Since ∂_n is the outward normal, D_n is signed according to which edge is being treated. Outward is taken to be negative at the left/bottom boundary and positive at the right/top, so D_n flips sign accordingly.

The discrete Higdon boundary condition can then be written using the matrix operators,

$$D_{tt}\mathbf{u}_b + (c_1 + c_2)D_{tn}\mathbf{u}_b + c_1 c_2 D_{nn}\mathbf{u}_b = 0, \quad (48)$$

where $\mathbf{u}_b$ are the boundary columns of the solution array.

D Reduction of the Gauss-Newton Method

Gauss-Newton is an iterative method for solving nonlinear least squares optimisation problems that approximates the Hessian of the objective function, avoiding the computation of second derivatives. At the k -th iterate, the residual is linearised as

$$\mathbf{r}(\mathbf{u}_k + \delta\mathbf{u}) \approx \mathbf{r}_k + J_k \delta\mathbf{u}, \quad (49)$$

where J_k is the Jacobian of the residual evaluated at $\mathbf{u}_k$. Minimising the resulting linearised least squares problem gives the Gauss-Newton normal equations,

$$J_k^T J_k \delta\mathbf{u}_k = -J_k^T \mathbf{r}_k. \quad (50)$$

For the present problem, however, the residual is linear in $\mathbf{u}$, such that $J_k = A$, where A is the constant global discretisation matrix of the all-at-once system. The Gauss-Newton equations therefore reduce to

$$A^T A \delta\mathbf{u}_k = -A^T \mathbf{r}_k. \quad (51)$$

Assuming that the discretised wave equation is uniquely solvable, such that A is non-singular, A^T is then also non-singular. Resultingly, Equation (50) can be written

$$A^T (A \delta\mathbf{u} + r) = 0. \quad (52)$$

since A^T is necessarily non-zero, the minimiser of the Gauss-Newton equations can be found by solving

$$A \delta\mathbf{u} = -r, \quad (53)$$

for the update, $\delta\mathbf{u}$. Using the definition of the residual $\mathbf{r}$, this can be written as

$$A(\mathbf{u}_k + \delta\mathbf{u}) = \mathbf{s}, \quad (54)$$

so a single update $\delta\mathbf{u}$ to the current iterate $\mathbf{u}_k$ gives the exact solution to the linear wave equation. This is a reflection of the property of the Gauss-Newton algorithm that it converges in exactly one outer iteration for linear problems [15].

E Diagonalisation of the α -circulant preconditioner

A circulant matrix $\tilde{D}_{tt}$ can be diagonalised by the eigendecomposition

$$\tilde{D}_{tt} = F^{-1} \Lambda F, \quad (55)$$

where F is the discrete Fourier transform matrix and Λ is a diagonal matrix containing the eigenvalues of $\tilde{D}_{tt}$. The ParaDiag II method uses an α circulant matrix whose wraparound block is scaled by some constant $\alpha \in (0, 1)$. As a result, the appropriate diagonalisation uses a weighted Fourier transform. With the diagonal scaling matrix $D_\alpha = \text{diag}(\alpha^{l/N_t})$ for $l = 0, \dots, N_t - 1$, the Fourier transform can be written

$$\tilde{D}_{tt} = D_{\alpha}^{-1} F^{-1} \Lambda_{\alpha} F D_{\alpha}, \quad (56)$$

where Λ_{α} is now diagonal with entries depending on α .

F Experimental Configurations

This section summarises the numerical and physical parameters used across each of the described experiments. Key parameters are summarised in Tables 2a and 2b, respectively, while a short discussion of other parameters follows below.

Experiment	Domain (m)	Duration (μ s)	N_x	N_y	N_t	DOF
Baseline	$[0, 0.1]^2$	113	68	68	231	1.07×10^6
Accuracy	$[0, 0.2]^2$	120	varied	varied	varied	varied
Serial performance	$[0, 0.2]^2$	226	varied	varied	varied	varied
Strong scaling	$[0, 0.2]^2$	226	135	135	553	1.01×10^7
Dispersion	$[0, 0.4]^2$	453	varied	varied	varied	varied
Stability (quantitative)	$[0, 0.1]^2$	113	88	88	varied	varied
Stability (qualitative)	$[0, 0.35]^2$	396	305	305	varied	varied

(a) Numerical discretisation parameters.

Experiment	Model	Source loc. (m)	Receiver config.	f_0 (kHz)
Baseline	Homogeneous	(0.05, 0.05)	$n = 1$, (0.08, 0.05) m	100
Accuracy	Homogeneous	(0.10, 0.10)	$n = 24$, halo*, $r = 0.045$ m	50
Serial performance	Inclusion*	(0.10, 0.10)	NA	varied
Strong scaling	Inclusion*	(0.10, 0.10)	NA	100
Dispersion	Homogeneous	(0.1, 0.2)	$n = 80$, 20 on 4 rays* $r \in [0.05, 0.15]$ m, $\theta \in [0^\circ, 45^\circ]$	50
Stability (quantitative)	Homogeneous	(0.05, 0.05)	$n = 1$, (0.08, 0.05) m	100
Stability (qualitative)	Shepp Logan Phantom*	(0.165, 0.20)	$n = 1$, (0.29, 0.05) m	100

(b) Physical and acquisition parameters.

Table 2: Computational parameters used in the described experiments. An NA entry signifies that a parameter is Not Applicable for the experiment it pertains to. * for more notes on model and receiver configurations, consult the discussion below.

Where referenced, the homogeneous model had $c = 1500 \text{ m s}^{-1}$. The so called ‘inclusion’ model featured a circular velocity anomaly of $c = 1800 \text{ m s}^{-1}$ against a background of $c = 1500 \text{ m s}^{-1}$, centred on the middle of the relevant domain with a radius of 0.03 m. The Shepp Logan Phantom model was derived from the scikit-image [36] API and was manipulated such that the skull had a wave speed of 2800 m s^{-1} , the brain tissue wave speed was in the range $[1500, 1600] \text{ m s}^{-1}$, and the background wave speed was 1480 m s^{-1} . Models are illustrated in Figure 3.

Where referenced in Table 2b, the ‘halo’ receiver configuration is comprised of an array of n receivers distributed evenly on circle with radius r centred on the middle of the domain. The

'rays' configuration is comprised of receivers distributed uniformly along 4 rays at angles of $0^\circ, 15^\circ, 30^\circ$, and 45° counterclockwise from the horizontal. The receivers sit at a distance r from the source location (s_x, s_y) . The location of the i -th receiver along the ray at angle θ_j is calculated according to

$$\begin{aligned}x_i &= s_x + r_i \cos \theta_j \\y_i &= s_y + r_i \sin \theta_j\end{aligned}$$

Other parameters include: $\alpha = 10^{-3}$ for the preconditioner, absolute and relative tolerances of 10^{-10} and 10^{-12} for the GMRES iterative solve, respectively, and a minimum number of Points Per Wavelength (PPW) of 10. For the stability experiments, finite difference stencils with time and space order of 2 were used. For the remaining experiments, a space order of 6 was used, with a time order of 2. All data were stored in double precision.

G Analytical Solution of the 2D Wave Equation

In a homogeneous domain, an analytical solution to the 2D wave equation with a transient forcing wavelet $s(t)$ can be obtained by convolving the source wavelet in time with the Green's function of the wave equation [37],

$$u(t, \mathbf{x}, \mathbf{x}_s) = s \star g = \int_0^t s(\tau) g(t - \tau, \mathbf{x}, \mathbf{x}_s) d\tau, \quad (57)$$

where $\mathbf{x}_s$ is the location of the source injection, $\mathbf{x}$ is the location of measurement, i.e., the receiver location, and τ is the time of observation.

First, recall the form of the time-domain Green's function of the two dimensional wave equation described in [13],

$$g(t - \tau, r) = \frac{1}{2\pi} \frac{H(t - \tau - r/c)}{\sqrt{(t - \tau)^2 - r^2/c^2}}. \quad (58)$$

where H is the Heaviside function, and $r = \|\mathbf{x} - \mathbf{x}_s\|$ is the spatial distance between a source-receiver pair.

The denominator of the Green's function can become singular, leading to instability in the discrete integration. Thus there is motivation to find an equivalent form of the convolution integral that avoids this singularity. In the following derivation, it is shown how the Green's function can be rewritten by means of a hyperbolic substitution.

With $a = r/c$, the time taken for a wave to move a distance r through a homogeneous medium with wave speed c , the Heaviside function reads

$$H(t - \tau - a) = \begin{cases} 1 & t - \tau - a > 0 \\ 0 & \text{otherwise} \end{cases} \quad (59)$$

and the requirement is

$$t - \tau - a > 0 \quad \Longleftrightarrow \quad \tau < t - a. \quad (60)$$

Thus, the upper integration limit reduces to $t - a$ up to which $H = 1$, leading to the explicit integral

$$u(r, t) = \frac{1}{2\pi} \int_0^{t-a} \frac{s(\tau)}{\sqrt{(t-\tau)^2 - a^2}} d\tau. \quad (61)$$

Let the duration be $\mathcal{T} = t - \tau$, then the differential and limits change:

$$\begin{aligned} \frac{d\mathcal{T}}{d\tau} = -1 &\implies d\mathcal{T} = -d\tau \\ \tau \in [0, t-a] &\implies \mathcal{T} \in [t, a] \end{aligned}$$

Making the substitution $\tau = t - \mathcal{T}$ and $d\tau = -d\mathcal{T}$, then

$$u(r, t) = \frac{1}{2\pi} \int_t^a \frac{s(t - \mathcal{T})}{\sqrt{\mathcal{T}^2 - a^2}} - d\mathcal{T}, \quad (62)$$

To make the interval increasing, interval is negated and the limits are flipped, cancelling the negative the differential introduced, yielding

$$u(r, t) = \frac{1}{2\pi} \int_a^t \frac{s(t - \mathcal{T})}{\sqrt{\mathcal{T}^2 - a^2}} d\mathcal{T}, \quad t > a. \quad (63)$$

The intuitive explanation of the limits here is that the Ricker wavelet and the Green's function are beign convolved between the arrival time $a = r/c$ and the current time t .

Now, a cosh substitution can be used to remove the singular denominator. Let $\mathcal{T} = a \cosh \nu$ for the dimensionless dummy integration variable $\nu : \nu \geq 0$. Then

$$\frac{d\mathcal{T}}{d\nu} = a \sinh(\nu) \implies d\mathcal{T} = a \sinh(\nu) d\nu, \quad (64)$$

$$\mathcal{T} \in [a, t] \implies \nu \in \left[\cosh^{-1} \left(\frac{a}{a} \right), \cosh^{-1} \left(\frac{t}{a} \right) \right] \quad (65)$$

$$= \left[0, \cosh^{-1} \left(\frac{t}{a} \right) \right] \quad (66)$$

and the denominator in the integral becomes

$$\sqrt{\mathcal{T}^2 - a^2} = \sqrt{a^2 \cosh^2(\nu) - a^2} = a \sqrt{\cosh^2(\nu) - 1} = a \sinh \nu, \quad (67)$$

by the identity $\cosh^2(x) - \sinh^2(x) \equiv 1$. Substituting in these results yields

$$u(r, t) = \frac{1}{2\pi} \int_0^{\cosh^{-1}(t/a)} \frac{s(t - a \cosh(\nu))}{a \sinh(\nu)} (a \sinh(\nu) d\nu), \quad (68)$$

which simplifies nicely down to the result stated in Equation (28):

$$u(r, t) = \frac{1}{2\pi} \int_0^{\cosh^{-1}(t/a)} s(t - a \cosh(\nu)) d\nu. \quad (69)$$

H Results Tables

Optimiser	$\bar{\mathcal{E}}$	Wall clock (s)	Iterations	Exit reason
LBFGBS	0.26854	222.6	1000	Iteration ceiling hit
GMRES (unpreconditioned)	1.41414	11.1	4	Tiny updates (crawling solve)
GMRES (preconditioned)	0.08261	4.8	1	Converged

Table 3: Results of the baseline measurements discussed in Section 6. $\mathcal{E}$ denotes the global error norm.

N_x, N_y	N_t	DOF	Δx	PPW	$\bar{\mathcal{E}}_{\text{ODIL}}$	$\bar{\mathcal{E}}_{\text{Devito}}$	GMRES iters.
80	136	870,400	0.002532	11.85	0.052191	0.053204	3
100	170	1,700,000	0.002020	14.85	0.034569	0.035088	3
120	203	2,923,200	0.001681	17.85	0.024245	0.024403	3
140	237	4,645,200	0.001439	20.85	0.018974	0.019090	3
160	271	6,937,600	0.001258	23.85	0.013713	0.013797	3
180	305	9,882,000	0.001117	26.85	0.010958	0.011150	3
200	339	13,560,000	0.001005	29.85	0.008472	0.008572	3

Table 4: Results of the accuracy sweep presented in Figure 4. $\bar{\mathcal{E}}_{\text{ODIL}}$ and $\bar{\mathcal{E}}_{\text{Devito}}$ denote the mean of the global error norm averaged over 3 repeated measurements. The GMRES iterations for the internal $A\delta u = -r$ solve in the Gauss-Newton methodology that ODIL employs are also reported.

f_0 (kHz)	DOF	Wall clock (s)			Peak RSS (GiB)		
		Devito	ODIL _{uc}	ODIL _c	Devito	ODIL _{uc}	ODIL _c
25	1.73×10^5	0.61	2.2	0.8	0.19	0.20	0.46
50	1.28×10^6	0.62	30.7	9.2	0.21	0.36	3.10
75	4.21×10^6	0.62	161.1	46.7	0.23	0.82	11.90
100	1.01×10^7	0.63	508.9	142.3	0.29	1.53	30.92
125	1.94×10^7	0.65	1316.4	365.7	0.39	2.71	65.16
150	3.33×10^7	0.68	2939.5	826.7	0.52	4.51	122.75
200	7.90×10^7	0.74	8884.2	2552.8	0.94	12.99	323.90

Table 5: Results of the serial performance measurements presented in Figure 5. nc and c denote 'non-caching' and 'caching', respectively.

Cores	ODIL (cached)			ODIL (uncached)		
	Wall (s)	Speedup	Eff. (%)	Wall (s)	Speedup	Eff. (%)
1	144.74	1.00	100.0	529.19	1.00	100.0
2	74.79	1.94	96.8	268.16	1.97	98.7
4	41.48	3.49	87.2	139.93	3.78	94.5
8	23.17	6.25	78.1	73.42	7.21	90.1
16	13.79	10.50	65.6	39.83	13.29	83.0
32	9.57	15.12	47.3	24.14	21.92	68.5
64	8.16	17.74	27.7	17.90	29.56	46.2
128	9.08	15.95	12.5	16.77	31.56	24.7

Table 6: Measured wall clock times, speedup factors, and scaling efficiencies (Eff.) for the strong scaling experiment presented in Figure 6. Results for Devito are omitted due to negligible performance gain, as documented.

CFL	g_{ODIL} (s^{-1})	R^2	$\bar{\mathcal{E}}_{\text{ODIL}}$	Dev. finite?
0.70	-1.02×10^4	0.44	0.169	✓
0.90	-9.73×10^3	0.41	0.134	✓
1.00	-7.73×10^3	0.41	0.112	✓
1.01	2.16×10^5	1.00	0.110	✓
1.03	2.25×10^5	0.99	0.106	×
1.05	2.59×10^5	0.99	0.100	×
1.10	2.26×10^5	0.99	0.113	×
1.30	2.20×10^5	0.98	0.157	×
2.00	2.37×10^5	0.97	0.362	×
3.00	2.50×10^5	0.98	0.726	×
5.00	3.07×10^5	0.99	1.416	×
9.20	2.29×10^5	0.99	1.440	×

Table 7: Results for the quantitative stability investigation presented in Figure 8. g_{ODIL} denotes the growth rate measured in the ODIL solution, along with the R^2 coefficient of determination for the fit. The mean ODIL error norm $\bar{\mathcal{E}}_{\text{ODIL}}$ measured over 3 repeats is reported. Finite Devito results were few, so instead an indicator of where Devito became non-finite is provided.